

PBR Assignment Guideline

3rd session: Generating cube maps

Introduction

To be able to accelerate our PBR technique, we will need to precompute all the information that we can. If you remember the rendering equation:

$$L_o(\mathbf{v}) = \int_{\Omega} f(\mathbf{l}, \mathbf{v}) \otimes L_i(\mathbf{l}) (\mathbf{n} \cdot \mathbf{l}) d\omega_i$$

To take into account the reflected light of a surface, we will need to compute the incoming light and the corresponding BRDF function for current light and view direction.

The point here is BRDF behaves differently for the specular and the diffuse reflections. So, we can split them into different terms:

$$f(l, v) = k_d * f_d + k_s * f_s$$

In the theory class, we see that one way to approximate the diffuse term of BRDF is using lambert:

$$f_{\text{Lambert}}(\mathbf{l}, \mathbf{v}) = \frac{\mathbf{c}_{\text{diff}}}{\pi}$$

$\mathbf{c}_{\text{diff}}$: fraction of light reflected, or diffuse color

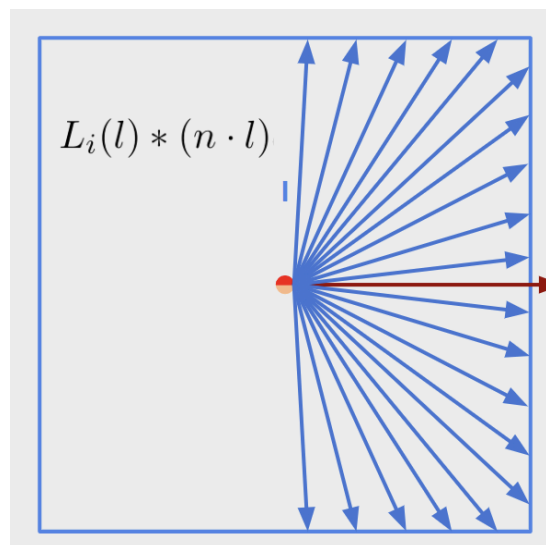
In short, Lambert proposes to integrate the incoming light for each direction of the hemisphere taking into account the angle between surface normal and incoming light so that the influence of the light is higher for perpendicular directions to the surface. As you can see, this term is completely independent of the viewer position, so we can put it out of the integral.

In our project to propose to use IBL as the lighting schema. IBL describes all the lighting of the scene as a unique big light source that surrounds everything. We proposed to accomplish it using a cube map that represents an environment map.

In this session, we will discuss how to generate a cube map that stores diffuse irradiance.

A short brief of theory

The IBL basis is that we can consider the scene as a single point. Following that, if we store all the irradiance (blue arrows) that a point receives for a determined normal (red arrow) in the position of the cubemap that this normal intersects it, obtaining the irradiance for each point of the surface is as simple as sample the cubemap in its normal direction.



Sampling the environment map to produce a diffuse irradiance map

Remember that the IBL schema provides us with a cubemap that will represent the radiance that surrounds the scene.

So, what we are going to do today is to generate the cubemap that will store the irradiance in any normal direction.

To achieve that, for any possible normal direction (i.e. texel of the cubemap) we will need to compute the hemisphere centered in this normal direction and compute the arriving irradiance as the integration of all the radiance that reaches our point from any direction of this hemisphere. I.e., we will need to sample the cubemap that stores the radiance of the environment for all the possible directions of the hemisphere. But sampling all the possible directions is too much, don't think so?

How to integrate the dome or a.k.a. Monte Carlo integration problem

So, how we can deal with this... In short, this is the typical Monte Carlo integration problem. We will sample a determined number of directions and we will consider that this set of samples will represent those directions that are not sampled. It will introduce some errors (mainly if the number of samples is low) but, in general, it works well.

To achieve it, we can mainly follow two different strategies: sampling randomly or sampling structured.

Sampling Randomly

We will perform several samples in random directions of the hemisphere and if they are enough and well distributed, we can expect a good representation of the irradiance. In this case, we can compute the irradiance as the average contribution of all the samples. Two points are important here and you will need to investigate a little bit about them (remember to explain your investigations and conclusions on the project report): randomness and quantity. For the quantity, you can try different options and use what satisfies your requirements. For the randomness, remember that we are talking about samples that need to be random in the hemisphere, so, not all random generators will obtain good results. Think that uniformly distributed random samples of the hemisphere are our goal! If they are not well distributed, you can be sampling more parts that are not important (parallel rays to the surface) than those more important (near the normal).

Structured sampling

Instead of sampling randomly, we can divide the hemisphere into regions and sample each region considering the sample to be representative of the whole region. In that case, we can't average all the samples, because each sample represents a region and each region can have different sizes. So, we will need to take into account the size of the region represented by each sample to integrate them. For that, you will need to use the solid angle of each region in this integration. In this case, the number of considered samples (or regions) will affect the final result. But you will have also some problems due to the repetitive sampling problem that you will need to investigate: remember to explain your investigations and conclusions in the project report.

In conclusion, we have different strategies to achieve the integration of the hemisphere. Each of them has its advantages and disadvantages, part of your work here is to deal with them: investigate them and decide how to solve them. Remember that it is important that the report reflects this learning process: so explain it and discuss it!

Besides that, there is a concept that is very interesting in Monte Carlo integration: importance sampling. Investigate it and think if it is possible to incorporate it into your computations. One of the mentioned alternatives is more straightforward.

After knowing what to do, we should know how to do it

How to solve this problem is completely open to your own decisions. I will explain to you different alternatives. However, it is your responsibility to choose one of them and justify it in the report (**YES, again, DON'T FORGET THE REPORT!**). Both of them will be considered correct, but you must be able to explain what are the advantages and disadvantages of each option.

Solve it in CPU

The most straightforward way to solve this problem is to solve it in the CPU. You can use any language or library (if that doesn't solve the whole problem). The key idea is to create a new image and for every pixel that corresponds to a texel of the cubemap, compute the irradiance of the corresponding normal direction sampling the environment map. After integrating all the samples, you store them in the image.

Solve it in GPU

There is an alternative to CPU that means taking advantage of the parallelism of the GPU. This can be done using a completely parallelized algorithm using CUDA (or any alternative). However, if you are not familiar with CUDA I strongly discourage that option as this language is completely away from the scope of the course. However, you can use the GPU in another way. As we are going to generate an image you can use OpenGL. You can render a QUAD for the whole viewport and, for each fragment, consider if it will be part of the cubemap. If it is the case, you will follow the same idea that in CPU. Compute the direction corresponding to the fragment, sample the cubemap that represents the environment map in several directions along the hemisphere, and integrate them.

As you can see, both alternatives are very similar. So, you are free to choose. However, in the end, you will need to have something like that:

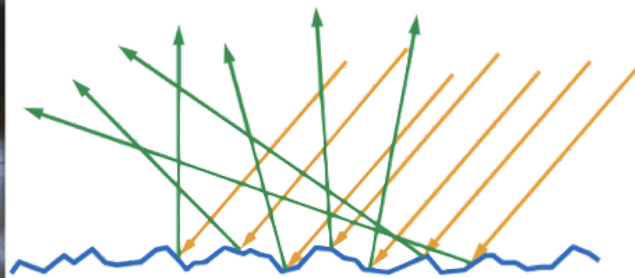
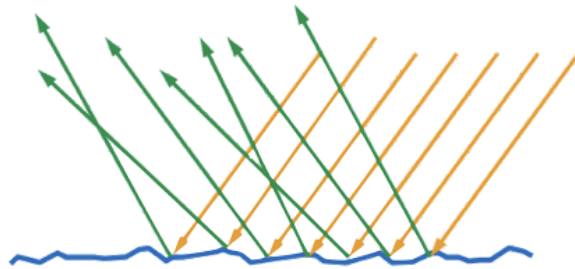


Original environment map

Computed Diffuse Irradiance Map

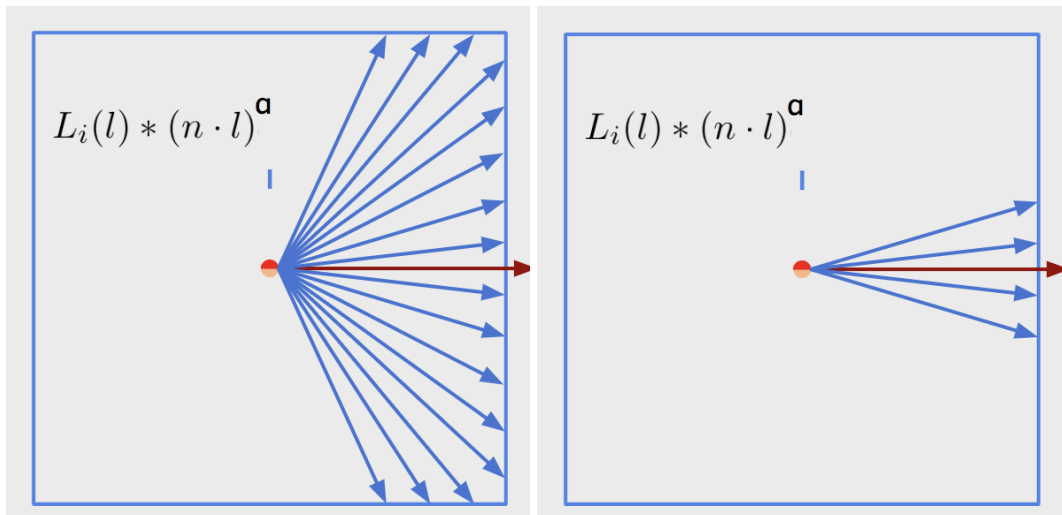
What about specular irradiance maps

For the specular irradiance maps, we can follow the same idea. However, there is a parameter that will play an important role in this computation: roughness. If you remember from theory, the roughness of the material affects how blurry the reflection is: if the material is rough, the region bounces reflections from different points, and the reflected image will be blurry; if the material is flat, all the bounces inside a region will reflect the same point the reflected image will be sharp.



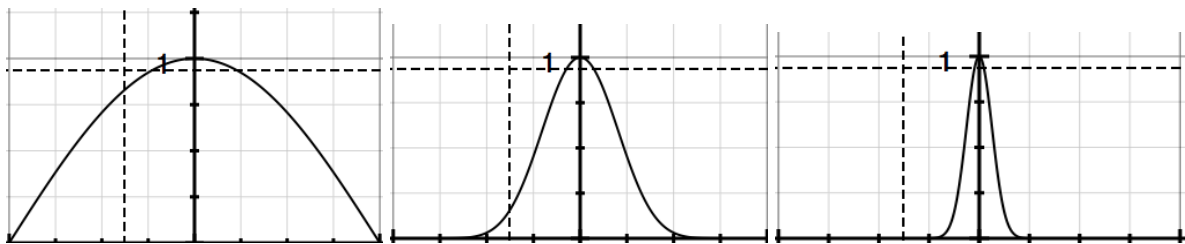
Flat surfaces (top) reflect sharp images, while rough surfaces (bottom) reflect blurry images.

One way to achieve that when we generate a specular irradiance map is to take into account the roughness of the material so the region of the hemisphere sampled will depend on the roughness: for rougher materials, we will sample a bigger region than for flat materials.



For rough materials (left) we sample a bigger part of the hemisphere than for flat materials (right)

We can achieve those following different strategies. One possibility is to apply importance sampling and sample only the interesting region. We can use the cosine between normal and sampling direction as the distribution function. Another possibility is to sample in all directions and use the inverse of roughness (i.e. $1/\text{roughness}$) to modify the cosine term in the integration increasing its power. By doing that, for rough materials all the samples will be considered but, for flat materials, samples far from the normal will not contribute to the lighting.



Contribution of the sample using $\cos(n \cdot l)^{(1/\text{roughness})}$ for different roughness values: 1 (left), 0.1 (center), and 0.01 (right)